Question 107 | Question bank page 10 | Q.2

Develop an E-Commerce PWA with product listing and offline cart functionality.

Project type: Progressive Web Application

Steps to perform

1. Create an ASP.NET Core MVC project or a suitable static web project in Visual Studio.

2. Add HTML or Razor pages, manifest.json, service-worker.js, and any referenced icons under wwwroot.

3. Register the service worker and link the manifest in the HTML layout.

4. Run the site on localhost or HTTPS and inspect it in a modern browser.

5. Use browser Developer Tools to check manifest, service worker, cache and offline operation. Push notifications require additional permission and subscription/server setup.

Closest supplied practical: app.UseStaticFiles(); app.Run(); Develop an E-Commerce PWA with product listing and offline cart functionality.

Source: Dot_Net_Practicals_6_to_10_Organized_v2.pdf

Provided creation instructions and source files

```
Creation steps: Create an ASP.NET Core Empty Web App named ECommercePWA. Add Program.cs,
manifest, product-listing page, service worker and app.js for cart storage.
Filename: Program.cs    Location: Program.cs
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.UseDefaultFiles();
app.UseStaticFiles();
app.Run();
Filename: Manifest.json    Location: wwwroot/manifest.json
{
  "name": "E-Commerce PWA",
  "short_name": "ShopPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
Filename: index.html    Location: wwwroot/index.html
<!DOCTYPE html>
<html>
<head>
    <title>E-Commerce PWA</title>
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">
    <style>
        body {
            font-family: Arial;
            margin: 20px;
        }
        .container {
            display: flex;
            gap: 30px;
        }
        .cart-section {
            width: 30%;
            border: 1px solid gray;
            padding: 15px;
            min-height: 300px;
        }
        .product-section {
            width: 70%;
```

```
        }
        .products {
            display: flex;
            flex-wrap: wrap;
            gap: 15px;
        }
        button {
            padding: 5px 10px;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <h1>E-Commerce PWA</h1>
    <div class="container">
        <div class="cart-section">
            <h2>Cart Items</h2>
            <ul id="cartList"></ul>
        </div>

        <div class="product-section">
            <h2>Products</h2>
            <div class="products">
                <div class="product">
                    <h3>Laptop</h3><p>Price: ■ 50000</p><button onclick="addToCart('Laptop')">
                        Add to Cart
                    </button>
                </div>
                <div class="product">
                    <h3>Mobile</h3><p>Price: ■ 20000</p><button onclick="addToCart('Mobile')">
                        Add to Cart
                    </button>
                </div>
                <div class="product">
                    <h3>He
adphones</h3><p>Price: ■ 3000</p><button onclick="addToCart('Headphones')">
                        Add to Cart
                    </button>
                </div>
            </div>
        </div>
    </div>
    <script src="app.js"></script>
</body>
</html>
Filename: service-worker.js    Location: wwwroot/service-worker.js
const CACHE_NAME = 'ecommerce-cache-v1';
const urlsToCache = [
    '/',
    '/index.html',
    '/app.js',
    '/manifest.json'
];
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
});
self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => {
                return response || fetch(event.request);
            })
    );
});
Filename: app.js    Location: wwwroot/app.js
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}
let cart = JSON.parse(localStorage.getItem('cart')) || [];
displayCart();
function addToCart(product) {
    cart.push(product);
    localStorage.setItem('cart', JSON.stringify(cart));
    displayCart();
}
function displayCart() {
```

```
    let cartList = document.getElementById('cartList');
    cartList.innerHTML = "";
    cart.forEach(item => {
        let li = document.createElement('li');
        li.textContent = item;
        cartList.appendChild(li);
    });
}
```

Expected result / verification

The application should meet the exact question requirements. Record actual output or screenshots after running; no execution has been claimed in this manual.